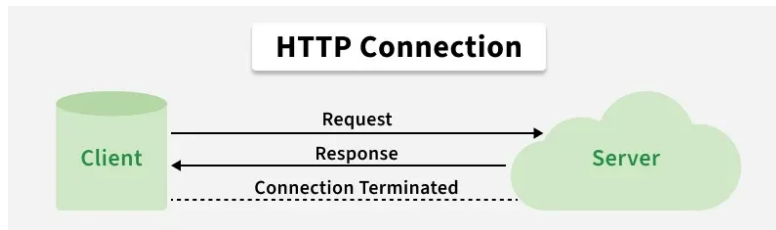


**HTTP (Hypertext Transfer Protocol):** HTTP is used not only for web browsing but also for machine-to-machine (M2M) communication, automation, and Internet of Things (IoT) applications.

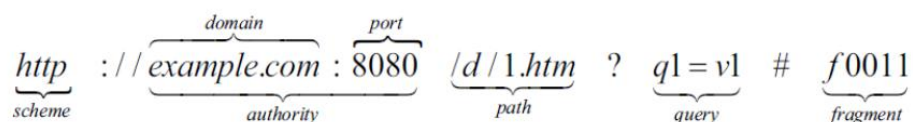
**Basics of HTTP:**



- 1.HTTP is a stateless request-response protocol in which a client sends a request and the server sends a response.
- 2.Each resource is identified using a URL, where the server is identified by the authority part and the resource by the path.
- 3.HTTP supports methods such as GET (retrieve data), POST (send data), PUT (update data), and DELETE (remove data).
- 4.HTTP works on top of IP and uses TCP for communication. TCP ensures reliable and ordered delivery of data. The default port is 80, and 8080 is commonly used.
- 5.For secure communication, SSL/TLS is used, and the protocol is called HTTPS, which normally uses port 443.

**Structure of a URL:** A URL (Uniform Resource Locator) is used to identify a resource on the Internet. It consists of the following parts:

- 1.**Protocol (Scheme):** Specifies the protocol used, such as http, https, or ftp.
- 2.**Authority:** Identifies the server and includes the domain name or IP address, and may include a port number.
- 3.**Path:** Specifies the location of the resource on the server.
- 4.**Query (Optional):** Provides additional information to the server in the form of key-value pairs.
- 5.**Fragment (Optional):** Points to a specific part of the resource.



Structure of a Uniform Resource Locator (URL)

**UPnP (Universal Plug and Play) Protocol:**

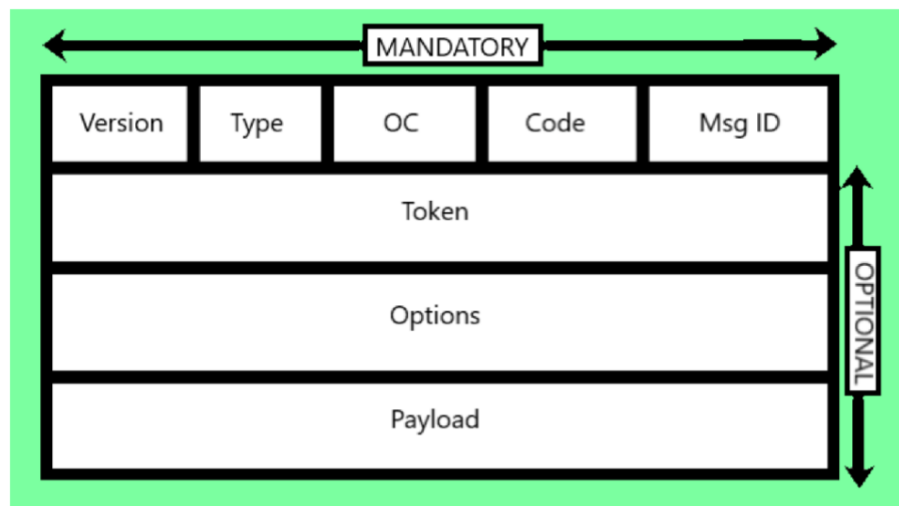
- 1.Universal Plug and Play (UPnP) is an architecture that allows devices in an ad hoc IP network to automatically discover each other and their services without manual configuration.
- 2.UPnP enables devices to discover services, perform actions, and receive event notifications, making device interaction easy and automatic.
- 3.UPnP is commonly used in home and office networks, especially in consumer electronics such as DLNA (Digital Living Network Alliance)-based devices.
- 4.UPnP uses HTTP over TCP and UDP, supporting both unicast (HTTPU) and multicast (HTTPMU) communication.
- 5.SSDP (Simple Service Discovery Protocol) is used for device discovery, and GENA (General Event Notification Architecture) is used for event subscription and notification.

6.Both SSDP and GENA introduce new HTTP methods to: search, notify and subscribe to and unsubscribe from an event.

### Constrained Application Protocol (CoAP):

- 1.CoAP is a lightweight application layer protocol designed for resource-constrained devices and networks, such as those used in the Internet of Things (IoT), where devices have limited processor, memory and power.
- 2.It is designed to be simple, low-overhead and energy-efficient, making it suitable for machine-to-machine communication.
- 3.CoAP follows a client-server model and is resource-oriented, where devices expose resources identified using URIs (Uniform Resource Identifier), similar to web services.
- 4.CoAP uses UDP instead of TCP, allowing asynchronous communication and reducing the need for acknowledgments, which helps save energy.
- 5.CoAP supports HTTP-like methods such as GET, POST, PUT and DELETE, enabling clients to retrieve, create, update and delete resources efficiently.

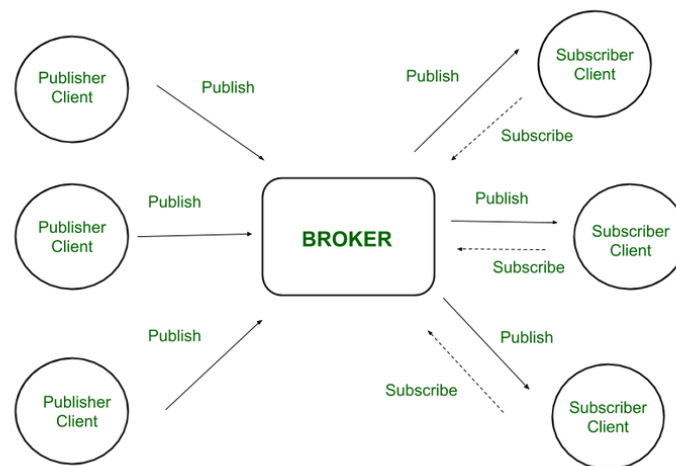
### Message Format of CoAP:



- 1.**Version:** The version field is 2 bits in size and indicates the CoAP protocol version being used.
- 2.**Type Code:** The type field is 2 bits long and defines the message type: Confirmable (00), Non-confirmable (01), Acknowledgement (10) and Reset (11).
- 3.**Option Count:** The option count field is 4 bits in size and specifies the number of options present in the message.
- 4.**Code:** The code field is 8 bits long and indicates whether the message is empty, a request, or a response.
- 5.**Message ID:** The message ID field is 16 bits in size and is used to identify messages and detect duplicates.
- 6.**Token:** The token field has a variable length (0–8 bytes) and is used to match a response with its request.
- 7.**Options:** The options field has a variable length and carries additional information related to the message.
- 8.**Payload:** The payload field has a variable size and contains the actual data of the request or response.

**Message Queue Telemetry Transport (MQTT) Protocol:**

1. MQTT is a lightweight communication protocol designed for Internet of Things (IoT) devices that work with low bandwidth, low power, and high network latency.
2. It is well suited for machine-to-machine (M2M) communication.
3. MQTT is a simple messaging protocol that allows multiple devices to exchange data efficiently.
4. It is a TCP-based protocol and uses the publish-subscribe model for communication.
5. Due to its low overhead and efficiency, MQTT is widely used in IoT frameworks for communication between resource-constrained devices.

**MQTT Client and Broker:**

1. In MQTT, clients publish messages on specific topics to a broker.
2. The broker acts as a central server that receives all messages from clients.
3. The broker filters messages based on topics and forwards them to clients that have subscribed to those topics.
4. Both publishers and subscribers are clients, and they never communicate directly with each other.
5. The broker can handle many connected clients at the same time and stores subscriptions and pending messages for persistent clients.
6. Hence, the broker is the heart of the MQTT publish-subscribe system.

**Advantages of MQTT:**

1. MQTT is a lightweight and simple protocol, suitable for IoT devices.
2. It uses small data packets, resulting in low bandwidth usage.
3. It provides efficient and fast message delivery.
4. MQTT supports remote sensing and control effectively.
5. It reduces power consumption, making it ideal for resource-constrained devices.
6. Data transmission is quick, efficient, and lightweight due to its small code footprint.

**XMPP (Extensible Messaging Presence Protocol):** XMPP is a communication protocol used to exchange messages and presence information over a network in near real time. It works by streaming XML elements between devices.

**Meaning of XMPP:**

- **X (Extensible):** XMPP is open and extensible, meaning it can be modified or extended as per requirements.
- **M (Messaging):** It is designed for real-time message delivery using an efficient push mechanism.
- **P (Presence):** It supports presence information such as online, offline or busy status.
- **P (Protocol):** XMPP is a standard protocol that enables systems to communicate with each other.

**Basic Functions of XMPP:**

- Send and receive messages between users.
- Share and check presence status.
- Manage contact lists and subscriptions.
- Control communication, such as blocking specific users.

**Features of XMPP:**

- XMPP follows a client-server architecture, where clients communicate through a server.
- It is decentralized, meaning anyone can run their own XMPP server, similar to email.
- Each user is identified using a Jabber ID (JID) in the format: username@domain/resource. The resource identifies different devices like mobile, desktop or web.

**Applications of XMPP:** XMPP is used in modified or customized form by applications such as WhatsApp, Kik Messenger and Zoom for messaging, presence and contact management.

**Internet of Things (IoT):** The Internet of Things (IoT) refers to a network of physical objects embedded with sensors, software and other technologies that enable them to connect and exchange data over the Internet. IoT includes a wide range of devices such as smart home appliances, industrial machines and agricultural sensors, which can collect, send and receive data automatically.

**An IoT system mainly consists of four components:**

- Sensors or devices that collect data from the environment,
- A network that connects the devices,
- A cloud platform that stores and processes the data, and
- A user interface that allows users to monitor and control the system.

**How IoT Devices Work:**

**1.Sensors and Devices:** IoT devices contain sensors that collect data from the environment such as temperature, moisture, or motion. Some devices also have actuators to perform actions like turning devices on or off.

**2.Connectivity:** The collected data is sent to the cloud through a network, which can be wired, wireless, or cellular.

**3.Data Processing:** In the cloud, the data is processed using software. The processing can be simple or complex and may involve machine learning algorithms.

**4.User Interface:** Users interact with the IoT system through a user interface such as a mobile app or web dashboard to view data, control devices, and receive alerts.

**Examples of IoT:**

**1.Smart Homes:** Devices such as smart thermostats, lights and appliances that can be controlled remotely.

**2.Wearable Technology:** Smartwatches and fitness trackers that monitor health and activity and connect to the Internet.

**3.Smart Agriculture:** Sensors used to monitor soil moisture, weather conditions and crop growth, helping to automate irrigation.

**4.Industrial IoT (IIoT):** Connected machines in factories that can be monitored, maintained and controlled remotely, supporting Industry 4.0.

**5.Connected Cars:** Vehicles equipped with sensors that provide real-time data for features such as driver alerts and navigation.

**Key Components and Characteristics of IoT:**

**1.Physical Objects:** Any physical object with an on/off switch can become an IoT device, such as appliances, machines, or sensors.

**2.Data Exchange:** The main function of IoT is to collect data and exchange it between devices and systems over a network.

**3.No Human Input:** IoT devices can communicate and take actions automatically without direct human involvement, while humans mainly monitor or control them.

**4.Embedded Technology:** IoT devices are built with processors, software, and sensors that allow them to sense, process, and transmit data.

**A simple IoT LED program:** A simple IoT LED program using ESP32 DevKit V1 involves connecting the board to Wi-Fi and sending LED status data to a cloud platform like ThingSpeak. The ESP32 controls an LED and uploads its ON/OFF status to the cloud for monitoring.

**Key Components:**

**1.Microcontroller:** ESP32 DevKit V1 (built-in Wi-Fi).

**2.LED:** Standard LED.

**3.Resistor:** 220–330 Ohm.

**4.Breadboard & Wires:** For circuit connection.

**5.Cloud Platform:** ThingSpeak.

**Steps (Using ESP32 & ThingSpeak):****1.Hardware Setup:**

- Connect LED positive leg + resistor to a GPIO pin (e.g., GPIO 2).
- Connect the negative leg to GND.
- GPIO → General Purpose Input Output pin.

**2.ThingSpeak Setup:**

- Create an account at ThingSpeak.com
- Create a new Channel
- Enable Field 1
- Copy your Write API Key

**3.Code (Arduino IDE):**

- Install ESP32 board package
- Install ThingSpeak library
- Enter your Wi-Fi credentials
- Enter ThingSpeak Write API Key
- Upload code to ESP32

**Overall: What the Program Does,**

- Connects ESP32 to Wi-Fi
- Connects to ThingSpeak cloud
- Turns LED ON and OFF automatically
- Sends LED status (0 or 1) to ThingSpeak Field 1
- Data can be viewed remotely on the ThingSpeak dashboard

**Simple LED CODE:**

```
int ledPin = 2;                      // GPIO2 (built-in LED on many ESP32 boards)
void setup() {
  pinMode(ledPin, OUTPUT);           // Set LED pin as output
}
void loop() {
  digitalWrite(ledPin, HIGH);        // Turn LED ON
  delay(1000);                       // Wait 1 second
  digitalWrite(ledPin, LOW);         // Turn LED OFF
  delay(1000);                       // Wait 1 second
}
```

**SenseIoT:** SenseIoT is an Internet of Things (IoT) system that uses sensors and connected devices to collect data from the environment and send it to the internet or cloud platform for monitoring and analysis. It helps in real-time data collection and remote monitoring of systems.

**Typically Does:**

- ❖ Sense environmental data using sensors (temperature, humidity, light, etc).
- ❖ Transmit the collected data to cloud platforms or servers.
- ❖ Process and display the data through dashboards or mobile apps.
- ❖ Trigger alerts or actions based on the sensed data.

**Common Use Cases:**

- **Smart agriculture:** Monitoring soil moisture and crop conditions
- **Smart homes:** Monitoring temperature, motion, and energy usage
- **Industrial monitoring:** Tracking machine conditions and performance
- **Environmental monitoring:** Measuring air quality, weather, or pollution levels
- **Healthcare monitoring:** Tracking patient health parameters

**Carriots:** Carriots is a cloud-based IoT platform that connects devices and sensors to the internet. It allows devices to send data to the cloud so users can monitor and control them remotely. It is commonly used in smart home, agriculture, and industrial IoT applications.

**Working of Carriots:**

- 1.Connectivity:** IoT devices and sensors connect to the Carriots cloud platform through the internet, allowing communication between devices and the cloud.
- 2.Collection:** The platform collects data sent from connected devices, such as sensor readings or device status.
- 3.Storage:** The collected data is stored in the cloud, where it can be accessed later for monitoring and analysis.
- 4.Dashboards:** The stored data is displayed on dashboards, which help users easily monitor devices and view real-time information.
- 5.Rules and Alerts:** Users can define rules and alerts so that the system automatically sends notifications or performs actions when certain conditions are met.

**Common Uses of Carriots:**

- 1.Smart Homes:** Used to monitor and control home devices such as lights, temperature sensors, and security systems.
- 2.Smart Agriculture:** Helps in monitoring soil moisture, temperature, and crop conditions using sensors.
- 3.Industrial Monitoring:** Used to track machine performance and equipment status in industries.
- 4.Environmental Monitoring:** Helps measure air quality, temperature, humidity, and weather conditions.
- 5.Energy Management:** Used to monitor energy consumption and improve power efficiency in buildings or industries.

**Node-RED:** Node-RED is a flow-based programming tool used to connect hardware devices, APIs, and online services in IoT applications. It provides a visual interface where users can create programs by connecting different blocks (nodes) together instead of writing complex code.

**Advantages of Node-RED:**

1. **Easy to Use:** Uses a visual drag-and-drop interface, making it simple for beginners.
2. **Fast Development:** Helps quickly build IoT applications and automation workflows.
3. **Integration:** Can easily connect sensors, devices, APIs, and cloud services.
4. **Open Source:** It is free and supported by a large community.

**Limitations of Node-RED:**

1. **Limited for Complex Systems:** Not ideal for very large or complex applications.
2. **Performance Issues:** May be slower when handling a large number of devices or data.
3. **Security Concerns:** Requires proper configuration to ensure secure communication.

**Working of Node-RED:**

- 1.Flow Creation:** Node-RED uses a visual flow-based interface where users create applications by connecting nodes (blocks) together.
- 2.Data Processing:** Each node performs a specific task such as receiving data, processing it, or sending it to another service.
- 3.Communication:** Node-RED can communicate with IoT devices, APIs, databases, and cloud platforms.

**4.Execution:** Once the flow is deployed, Node-RED automatically processes incoming data and performs actions based on the defined flow.

**Common Uses of Node-RED:**

- 1.IoT Device Integration:** Connecting sensors, actuators, and IoT devices.
- 2.Home Automation:** Controlling smart home devices like lights and thermostats.
- 3.Data Visualization:** Sending sensor data to dashboards for monitoring.
- 4.Industrial Monitoring:** Monitoring machines and industrial equipment.
- 5.Automation Workflows:** Creating automated processes based on data conditions.

**Clayster:** Clayster is an IoT platform used to connect devices and sensors to the internet. It helps collect, manage, and monitor data from IoT devices through the cloud.

**Why Clayster is Needed:**

Clayster is needed to simplify the development and management of IoT systems. It provides a platform where multiple devices can connect, exchange data, and be monitored from a central location. This makes it easier to handle large numbers of IoT devices and automate tasks.

**Main Features of Clayster:**

- 1.Device Connectivity:** Allows IoT devices and sensors to connect to the platform through the internet.
- 2.Data Collection:** Collects data from different connected devices and sensors.
- 3.Data Storage:** Stores the collected data in the cloud for monitoring and analysis.
- 4.Real-Time Monitoring:** Enables users to monitor device data and system status in real time.
- 5.Security:** Provides authentication and security to protect connected devices and data.

**Working of Clayster:**

- 1.Device Connection:** IoT devices and sensors connect to the Clayster platform through the internet.
- 2.Data Transmission:** Devices send sensor data to the cloud platform.
- 3.Data Processing:** The platform processes and stores the received data.
- 4.Visualization:** The data is displayed on dashboards for monitoring.
- 5.Control and Alerts:** Users can monitor devices and set rules or alerts based on the data.

**Example of Clayster:** In smart agriculture, sensors measure soil moisture and temperature. The data is sent to Clayster, where farmers can monitor the conditions and control irrigation remotely.

**Thingier.io:** Thingier.io is an open-source cloud IoT platform that allows devices and sensors to connect to the internet and communicate with cloud services. It helps users collect, manage, store, and visualize data from IoT devices through dashboards and web applications.

**Working of Thingier.io:**

- 1.Device Connection:** IoT devices such as ESP32, ESP8266, Arduino, or Raspberry Pi connect to the Thingier.io cloud platform through the internet.
- 2.Data Transmission:** The connected devices send sensor data to the Thingier.io server using supported communication protocols.



**3.Data Processing and Storage:** The platform processes and stores the received data in the cloud for monitoring and analysis.

**4.Visualization:** The data is displayed on dashboards, allowing users to view sensor values and system status in real time.

**5.Control and Automation:** Users can control devices remotely and create automation rules to perform actions based on sensor data.

#### **Main Features of Thingier.io:**

**1.Device Connectivity:** Allows IoT devices like ESP32, Arduino, and Raspberry Pi to connect easily to the cloud.

**2.Real-Time Data Monitoring:** Provides real-time visualization of sensor data through dashboards.

**3.Data Storage:** Stores device data in the cloud for later analysis and monitoring.

**4.Remote Control:** Enables users to control IoT devices remotely through web dashboards.

**5.Integration Support:** Supports integration with different IoT hardware, APIs, and services for building complete IoT applications.

#### **Advantages of Thingier.io:**

**1.Free and Open Source:** Thingier.io is free to use and open source, allowing developers to modify and customize the platform.

**2.Simple but Powerful:** It provides an easy interface for building IoT applications while offering powerful tools for device management and data monitoring.

**3.Hardware Agnostic:** Thingier.io supports many types of IoT hardware such as ESP32, Arduino, and Raspberry Pi, making it flexible for different projects.

**4.Scalable and Efficient:** The platform can handle many devices and large amounts of data efficiently, making it suitable for both small and large IoT systems.

**IoT Security:** IoT security refers to the protection of IoT devices, networks, and data from cyber threats and unauthorized access. Since IoT devices are connected to the internet, they are vulnerable to attacks and require proper security mechanisms.

#### **Security Challenges:**

**1.Inadequate Device Security:** Many IoT devices have weak security features, making them easy targets for attackers.

**2.Unsecured Data Transmission:** Data sent between devices may not be properly encrypted, which can lead to data theft.

**3.Privacy Concerns:** IoT devices collect sensitive user data which may be misused if not properly protected.

**4.Vulnerability to Network Attacks:** IoT systems can be attacked through the network, affecting devices and services.

**Interoperability Challenges:** Interoperability refers to the ability of different IoT devices and systems to communicate and work together effectively.

**Interoperability Challenges:**

**1.Lack of Standardization:** Different manufacturers use different communication protocols and standards.

**2.Heterogeneity of Devices:** IoT networks contain devices with different hardware, software, and capabilities.

**3.Scalability Issues:** As the number of devices increases, managing communication between them becomes difficult.

**Risks in IoT:** IoT systems face several risks because of their connectivity and distributed nature.

**Major Risks:**

**1.Weak Authentication:** Poor authentication mechanisms allow unauthorized users to access devices.

**2.Data Privacy Breaches:** Sensitive information collected by IoT devices may be exposed.

**3.Insecure Communication:** Data transmitted without encryption can be intercepted by attackers.

**4.Botnet Attacks:** Compromised IoT devices can be used to launch large-scale attacks.

**5.Poor Update Mechanism:** Lack of regular software updates can leave devices vulnerable.

**6.Physical Tampering:** Attackers may physically access devices and manipulate them.

**Modes of Attacks in IoT:** IoT systems can face different types of cyber attacks.

**Types of Attacks:**

**1.Malware Attacks:** Malicious software is used to damage or control IoT devices.

**2.Botnet Attacks:** Multiple compromised devices are controlled by attackers to launch large-scale attacks.

**3.Denial of Service (DoS / DDoS):** Attackers overload the network or system so legitimate users cannot access services.

**4.Firmware Attacks:** Attackers modify or replace device firmware to gain control over the device.

**5.Physical Attacks:** Attackers gain direct physical access to IoT devices to manipulate them.

**6.Replay Attacks:** Previously transmitted data packets are captured and retransmitted to trick the system.

**Man-in-the-Middle (MITM) Attack:** A Man-in-the-Middle attack occurs when an attacker secretly intercepts communication between two devices. The attacker can monitor, modify, or steal the transmitted data without the knowledge of the communicating parties.

**Spoofing Attack:** In a spoofing attack, the attacker pretends to be a trusted device or user to gain unauthorized access to the system.

**Password Attacks:** Password attacks involve attempts to guess or steal passwords to gain access to IoT devices or systems.

**Eavesdropping Attack:** Eavesdropping occurs when an attacker secretly listens to the communication between devices to obtain sensitive information.

**Tools for IoT Security:** IoT security tools are used to protect IoT devices, networks, and data from cyber threats and unauthorized access. These tools help ensure secure communication, device authentication, and protection against attacks.

**1.Encryption Tools:** Encryption tools are used to protect data during transmission by converting it into an unreadable format. This ensures that only authorized users can access the information.

**2.Authentication and Authorization Tools:** These tools verify the identity of devices and users before allowing access to IoT systems. They also control what actions the authenticated user is allowed to perform.

**3.Firewalls and Network Security Tools:** Firewalls help monitor and control network traffic and prevent unauthorized access to IoT networks.

**4.Intrusion Detection and Prevention Systems (IDS/IPS):** These systems detect and prevent malicious activities or attacks in IoT networks by monitoring network behavior.

**5.Secure Firmware Update Tools:** These tools ensure that software updates for IoT devices are secure and authentic, preventing attackers from installing malicious firmware.

**6.Device Management Tools:** Device management tools help monitor, configure, and control IoT devices remotely to ensure proper functioning and security.

**7.Key Management Tools:** Key management tools are used to generate, store, and manage encryption keys securely, which are essential for secure communication in IoT systems.

**Tools for IoT Interoperability:** IoT interoperability tools help different IoT devices, platforms, and systems communicate and work together, even if they are built by different manufacturers or use different technologies.

**1.Middleware Platforms:** Middleware acts as a bridge between IoT devices and applications, allowing devices with different protocols and technologies to communicate with each other.

**2.Standard Communication Protocols:** Standard protocols such as MQTT, CoAP, HTTP, and Zigbee help different IoT devices exchange data using common communication standards.

**3.IoT Gateways:** IoT gateways connect local IoT devices to cloud platforms and translate different communication protocols so devices can work together.

**4.APIs (Application Programming Interfaces):** APIs allow different IoT applications and services to exchange data and integrate with each other easily.

**5.Data Standardization Tools:** These tools ensure that data from different devices is formatted in a common structure, making it easier to process and share across systems.

**6.Cloud IoT Platforms:** Cloud platforms such as AWS IoT, Azure IoT, and Thingier.io help manage devices, store data, and enable communication between different IoT systems.